AEGISAI ENTERPRISE EDITION

TECHNICAL DOCUMENTATION SUITE

Administration & Operations Manual

DevSecOps, Secrets Governance, Quota Management & Observability

Platform Version:	**2.5.0 Enterprise Edition**
Compliance Scope:	**OWASP AISVS v1.0, ISO/IEC 42001:2023, Google Cloud Security**
License:	**Apache License 2.0 (Open Source)**

AEGIS AI ASSURANCE PLATFORM

Administration and Operations Manual (DevSecOps Guide)

Version: 2.5.0 Enterprise (2026 Edition)

Classification: Operational Technical Guide / General Use

Scope: Platform Governance, Key Management, Observability, and Operations

---

1. Administrator Profile and Responsibilities

The AegisAI Administrator is responsible for:

1. Custodying and rotating system master credentials (`ADMIN_API_TOKEN` and `GEMINI_API_KEY`).
2. Monitoring quota, latency, and inference telemetry for the Google Gemini API.
3. Managing the custom regulatory frameworks catalog (BYOF module) and authorizing the publication or deprecation of organizational regulations.
4. Maintaining the integrity and availability of the dual-storage layer (local JSON files and Cloud Firestore database).
5. Handling operational incidents and ensuring compliance with Service Level Agreements (SLAs).

---

2. Identity Management and Access Control

2.1 Layered Authentication Scheme

AegisAI implements three complementary authentication mechanisms:

Access Level	Mechanism	Variable / Header	Operational Scope
Web Administration	Session Secret Token	`x-admin-token` header	Access to `/api/admin/settings`, `/api/admin/test-connection`, and framework deletion.
CI/CD Automation	Service API Key	`x-api-key` header	Invoking asynchronous audits via `/api/v1/audits` from GitHub Actions or GitLab CI.
Local Operator	Emergency Session	Bootstrap token	Access to the user interface in standalone mode when Firebase Auth is unavailable.

2.2 Administrative Authentication Middleware

The `/api/admin/*` endpoint is protected by the `authenticateAdmin` middleware in `server.ts` :

```
const authenticateAdmin = (req: express.Request, res: express.Response, next: express.NextFunction)
  const configuredToken = process.env.ADMIN_API_TOKEN;
  const providedToken = req.headers['x-admin-token'] ||
    (req.headers.authorization?.startsWith('Bearer ') ? req.headers.authorization.split(' ')[1] : nu

  if (configuredToken && providedToken === configuredToken) {
    return next();
  }
  return res.status(401).json({ error: 'Unauthorized access: Invalid or missing administration toker
};
```

3. Multi-Model Artificial Intelligence Engine Configuration and Monitoring

3.1 Decoupled Multi-Provider Support

AegisAI features a universal decoupled inference engine that allows operating with both Google Gemini models and external providers/routers compatible with the OpenAI specification (`/v1/chat/completions`):

- **Google Gemini:** Native models (`gemini-3.8-flash` , `gemini-3.1-flash-lite` , `gemini-3.5-pro`) via the `@google/genai` SDK with tolerance for demand spikes.
- **OpenRouter (Universal Router):** Centralized routing to Claude 3.7 Sonnet, GPT-4o, DeepSeek R1, Llama 3.3 70B, etc., with `HTTP-Referer` and `X-Title` headers.
- **Direct OpenAI:** Flagship OpenAI models (`gpt-4o` , `gpt-4o-mini` , `o1` , `o3-mini`).
- **Local / Universal Proxy (OpenAI-Compatible):** For private on-premise inference using **vLLM**, **Ollama**, **LiteLLM**, **OpenWebUI**, or **LM Studio**.

3.2 Environment Variables and File-Based Configuration

Engine options are parameterized in the `.env` file or in `config/admin-settings.json` :

```
# Google Gemini API Key (required if using the Gemini provider)
GEMINI_API_KEY="AIzaSy..."

# Administrative master token for the configuration panel
ADMIN_API_TOKEN="your_secure_admin_token"

# Port and environment
PORT=3000
NODE_ENV="production"
```

Dynamic persistence structure in `config/admin-settings.json` :

```
{
  "provider": "openrouter",
  "preferredModel": "gemini-3.8-flash",
  "openaiApiKey": "sk-or-v1-xxxxxxxxxxxx",
  "openaiBaseUrl": "https://openrouter.ai/api/v1",
  "openaiModel": "anthropic/claude-3.7-sonnet",
  "customHeaders": {
    "HTTP-Referer": "https://aegisai.local",
    "X-Title": "AegisAI Compliance"
  },
  "temperature": 0.0,
  "maxTokens": 8192,
  "fallbackModel": "openai/gpt-4o-mini",
  "updatedAt": "2026-09-17T14:40:00.000Z"
}
```

3.3 Inference Control and Governance Parameters

1. **Temperature:** Configurable from 0.0 to 1.0 (Default: `0.0` to ensure deterministic and scientifically reproducible compliance evaluations).
2. **Output Token Limit (`maxTokens`):** Maximum generation limit per evaluation block (Default: `8192`).
3. **Fallback Model (`fallbackModel`):** Secondary model that takes over the load if the primary model suffers temporary quota exhaustion (HTTP 429/503).
4. **Custom Headers (`customHeaders`):** JSON object to inject corporate headers, billing tags, or custom authentication.

3.4 Real-Time Connectivity Diagnostics and Hot-Reload

From the web administration interface or via the REST API, the administrator can validate the inference channel by measuring **latency in milliseconds (ms)**:

```
curl -X POST http://localhost:3000/api/admin/test-connection \
  -H "Content-Type: application/json" \
  -H "x-admin-token: YOUR_ADMIN_API_TOKEN" \
  -d '{"provider": "openrouter", "model": "anthropic/claude-3.7-sonnet"}'
```

Expected response:

```
{
  "ok": true,
  "provider": "openrouter",
  "latencyMs": 1150,
  "model": "anthropic/claude-3.7-sonnet",
  "message": "Successful connection with OPENROUTER API",
  "sampleResponse": "OK"
}
```

When saving in the UI, changes are applied immediately via **hot-reload** without requiring a restart of the Node.js server or the container.

3.5 LLM Engine Automated Test Suite

To certify the correct operation of the providers:

```
npm run test:llm
```

Executes 6 automated tests: initial state verification, live test with latency measurement, dynamic OpenRouter/OpenAI configuration, persistence check with key obfuscation, fault tolerance against network drops, and deterministic restoration.

---

4. Dual Storage Management and Persistence

4.1 Dual Storage Architecture

To ensure immunity against network issues or cloud security rule restrictions, AegisAI implements hybrid persistence:

1. **Local JSON Directory (`config/frameworks/*.json`):** Master storage on disk for ingested regulations (LFPDPPP, LFPC, EU AI Act).
2. **Cloud Firestore Collection (`custom_frameworks` and `audits`):** Cloud replication for multi-instance synchronization.

4.2 Framework Catalog Backups

To perform a preventative backup of all laws and policies in the BYOF module:

```
# Create a timestamped backup directory
mkdir -p /backups/aegisai/$(date +%Y%m%d)

# Copy the entire configurations and frameworks directory
cp -r /opt/aegisai/config/frameworks/*.json /backups/aegisai/$(date +%Y%m%d)/
```

4.3 Procedure for Deleting a Regulatory Framework

To decommission an obsolete regulation:

```
curl -X DELETE http://localhost:3000/api/frameworks/fw-id-to-delete \
  -H "x-admin-token: YOUR_ADMIN_API_TOKEN"
```

The system will delete the local JSON file and remove the corresponding document in Cloud Firestore.

---

5. Telemetry, Logs, and Observability

5.1 Per-Audit Metrics Logging

Each processed audit formally records the following AI telemetry attributes in its database document:

- `modelUsed` : The exact Gemini model that generated the verdict (e.g., `gemini-3.1-flash-lite`).
- `latencyMs` : Total engine response time in milliseconds.
- `promptTokens` : Total tokens consumed in the input specifications.
- `completionTokens` : Total tokens generated in the analyses and mitigation recipes.

5.2 Log Sanitization and Shielding

The server features suppression filters that ensure no API keys (`AIzaSy...`), administrative tokens, or sensitive client information appear in console logs or event forwarders.

5.3 Application Health Monitoring

Probing endpoint for load balancers and Kubernetes probes (Liveness / Readiness Probes):

```
curl -I http://localhost:3000/api/health
# Response: HTTP/1.1 200 OK -> {"status":"ok"}
```

---

6. Standard Operating Procedures (SOPs)

SOP-01: Google Gemini API Key Rotation

1. Generate a new API key in the Google Cloud Console / Google AI Studio.
2. Edit the `.env` file on the server:

   ```
   nano .env
   # Update the GEMINI_API_KEY="NEW_KEY" variable
   ```

3. Restart the AegisAI service:

   ```
   systemctl restart aegisai
   ```

4. Run the connectivity test to verify proper operation:

   ```
   curl -X POST http://localhost:3000/api/admin/test-connection -H "x-admin-token: YOUR_TOKEN"
   ```

SOP-02: Exceeded Quota Incident Response (HTTP 429)

1. Verify consumption in the Google Cloud APIs & Services console.
2. If the project has reduced requests-per-minute (RPM) quota limits, configure the alternative model `gemini-3.1-flash-lite` in `.env` or the administration view, as it features the highest limits and lowest cost per token.
3. Submit a Quota Increase Request in the Google Cloud Console for the Gemini v1beta API.